

사전 (Dictionary)

강 지 훈

jhkang@cnu.ac.kr



사전 (Dictionary) 이란?

□ 개요

■ 사전이란?

- (단어, 설명) 의 쌍들을 모아놓은 것.
- 단어가 주어지면 그 단어의 설명을 찾는다.

■ 또 다른 예는?

- (학번, 성적)

■ 일반화 하면?

- (key, object) 의 쌍을 모아 놓은 집합.
- Key 가 주어지면, 그에 해당하는 object 를 찾는다.
- key 값은 유일하다.

Class “Dictionary”

□ Dictionary 의 공개함수

■ Dictionary 객체 사용법

- `public Dictionary()` ;
- `public boolean isEmpty ()` ;
- `public boolean isFull ()` ;
- `public int size()` ;
- `public boolean keyDoesExist (Key aKey)` ;
 - ◆ 주어진 aKey 가 사전에 존재하는지 여부를 확인한다
- `public Object objectForKey (Key aKey)` ;
 - ◆ 주어진 aKey 를 갖는 객체를 얻는다
- `public boolean addKeyAndObject (Key aKey, Obj anObject)` ;
 - ◆ <aKey, anObject> 쌍을 사전에 추가한다.
- `public Obj removeObjectForKey (Key aKey)` ;
 - ◆ 주어진 aKey와 쌍을 이루는 객체를 함께 삭제한다.
- `public boolean replaceObjectForKey (Key aKey, Obj objectForReplace)` ;
 - ◆ 주어진 aKey 와 쌍을 이루는 객체를 새로운 objectForReplace 로 대체한다.
- `public void clear ()` ;
 - ◆ 사전의 모든 <key, object> 쌍을 삭제하여, 사전을 비운다.

이진검색트리 (Binary Search Trees)

이진 검색 트리

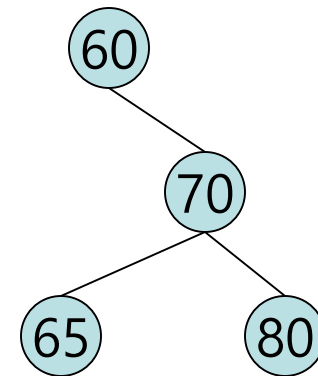
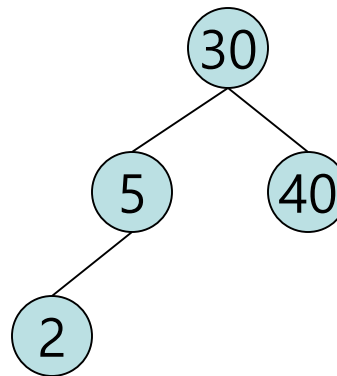
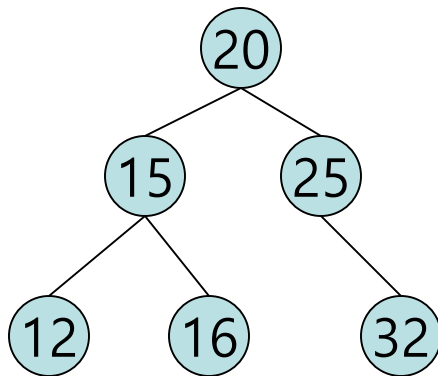
임의의 원소의 삽입/삭제/검색

시간복잡도 비교

	정렬되지 않은 배열	정렬된 배열	이진검색트리
삽입	$O(1)$	$O(n)$	$O(\log n)$
삭제	$O(n)$	$O(n)$	$O(\log n)$
검색	$O(n)$	$O(\log n)$	$O(\log n)$

이진검색트리 (Binary Search Trees)

- **이진검색트리**는 이진트리이다. 비어 있을 수 있으며, 만일 비어 있지 않다면 다음의 조건을 만족해야 한다:
 1. 모든 원소는 키를 가지고 있으며, 어떠한 원소도 동일한 키를 가지고 있지 않다. 즉, **키는 유일(unique)하다**.
 2. 트리의 루트의 키는 비어있지 않은 왼쪽 부트리에 있는 키들보다 **크다**.
 3. 트리의 루트의 키는 비어있지 않은 오른쪽 부트리에 있는 키들보다 **작다**.
 4. 왼쪽 부트리와 오른쪽 부트리는 또한 이진검색트리이다.



□ 검색 알고리즘: [재귀적으로]

```

BinaryNode<E> search (BinaryNode<E> currentRoot, int keyForSearch)
{
    if ( currentRoot != null ) {
        if ( keyForSearch.compareTo(currentRoot.element().key()) == 0 )
            return currentRoot ;
        else if ( keyForSearch.compareTo(currentRoot.element().key()) < 0 )
            return search (currentRoot.left(), keyForSearch) ;
        else
            return search (currentRoot.right(), keyForSearch) ;
    }
    else {
        return null ;
    }
}

```

- 검색 시간 복잡도: $O(h)$
- h: 이진검색트리의 높이

```

public class Element<Key, Obj> {
    private Key _key ;
    ..... // 다른 변수들
    public Key key() {...} ;
    public void setKey() {...} ;
    ..... // 다른 공개함수들
}

public class BinaryNode<E> {
    private Element<Key,Obj> _element;
    private BinaryNode<E> _left;
    private BinaryNode<E> _right ;

    public E    element() {...} ;
    public void    setElement() {...} ;
    public BinaryNode<E> left() {...} ;
    public void    setLeft() {...} ;
    public BinaryNode<E> right() {...} ;
    public void    setRight() {...} ;
}

```

□ 검색 알고리즘: [반복적으로]

```

BinaryNode<E> search (BinaryNode<E> currentRoot, int keyForSearch)
{
    while ( currentRoot != null ) {
        if ( keyForSearch.compareTo(currentRoot.element().key()) == 0 )
            return currentRoot ;
        else if ( keyForSearch.compareTo(currentRoot.element().key()) < 0 )
            currentRoot = currentRoot.left() ;
        else
            currentRoot = currentRoot.right() ;
    }
    return null ;
}

```

■ 검색 시간 복잡도: $O(h)$

- h: 이진검색트리의 높이

```

public class Element<Key, Obj> {
    private Key _key ;
    ..... // 다른 변수들
    public Key key() {...} ;
    public void setKey() {...} ;
    ..... // 다른 공개함수들
}

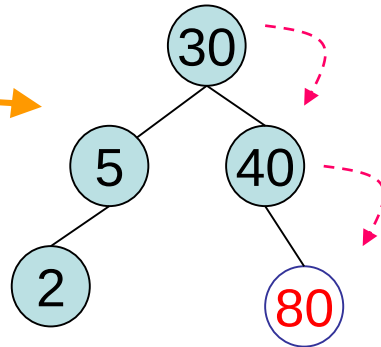
public class BinaryNode<E> {
    private Element<Key,Obj> _element;
    private BinaryNode<E> _left;
    private BinaryNode<E> _right ;

    public E element() {...} ;
    public void setElement() {...} ;
    public BinaryNode<E> left() {...} ;
    public void setLeft() {...} ;
    public BinaryNode<E> right() {...} ;
    public void setRight() {...} ;
}

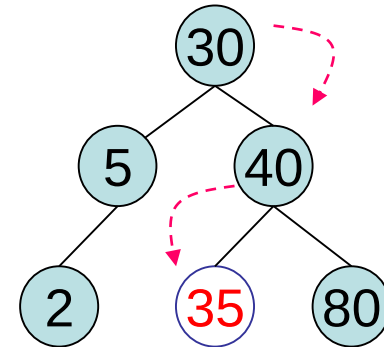
```

삽입

Insert 80



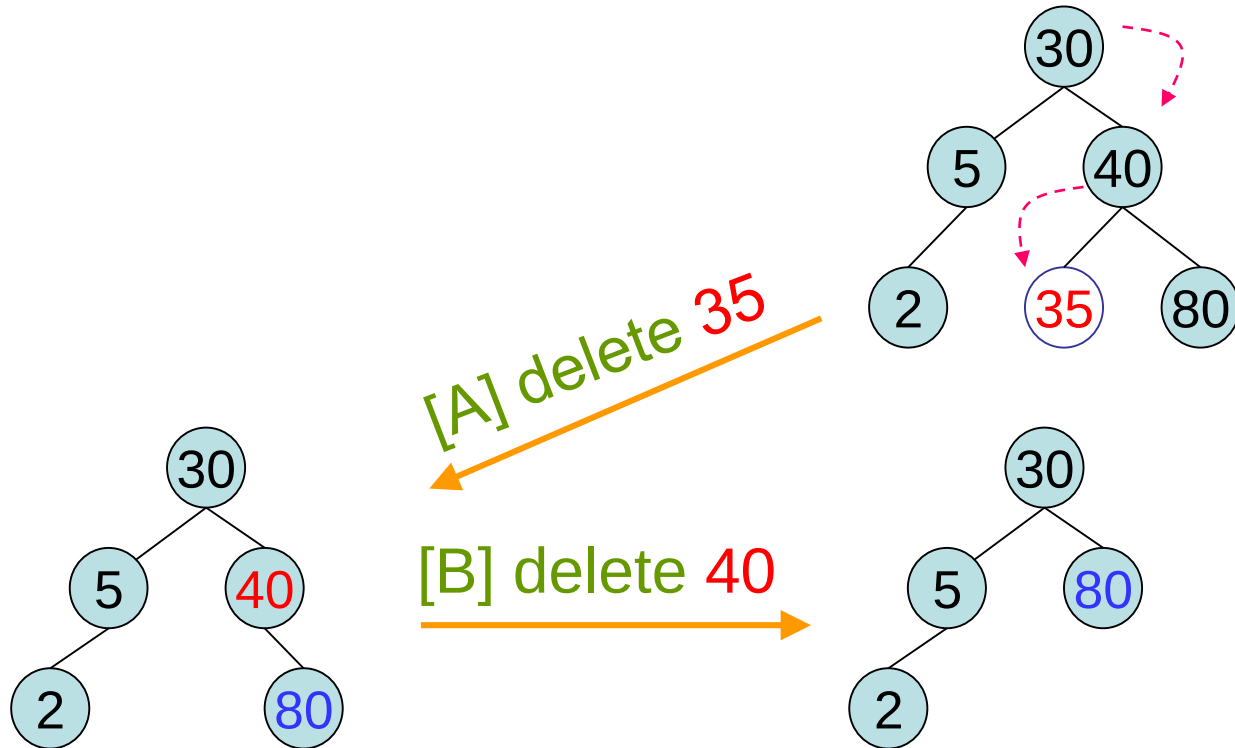
Insert 35



■ 삽입의 시간복잡도: $O(h)$

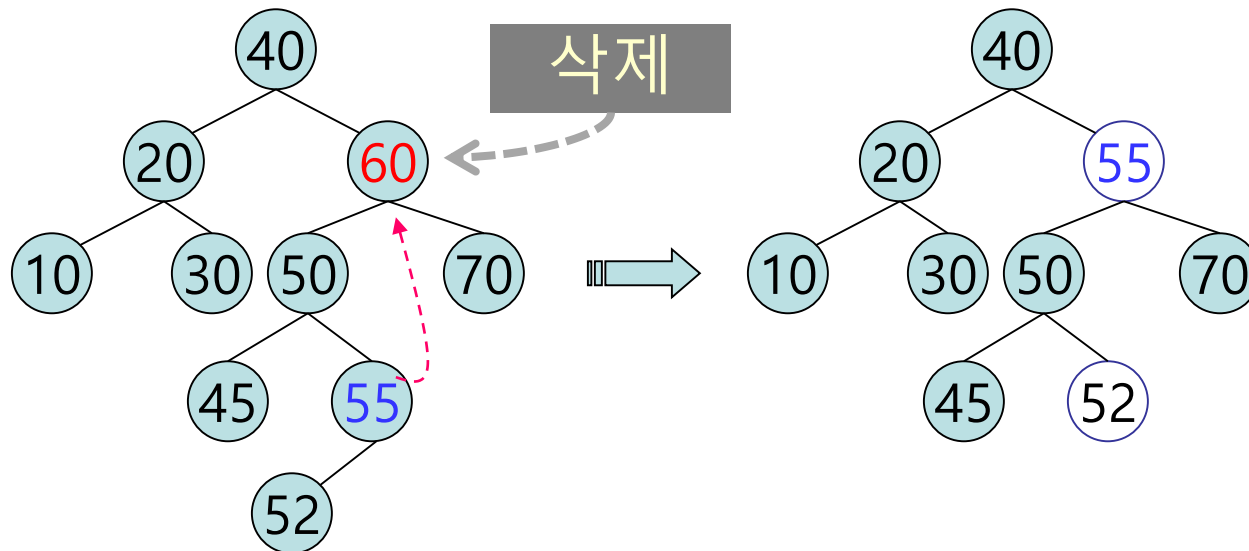
● h : 이진검색트리의 높이

□ 삭제



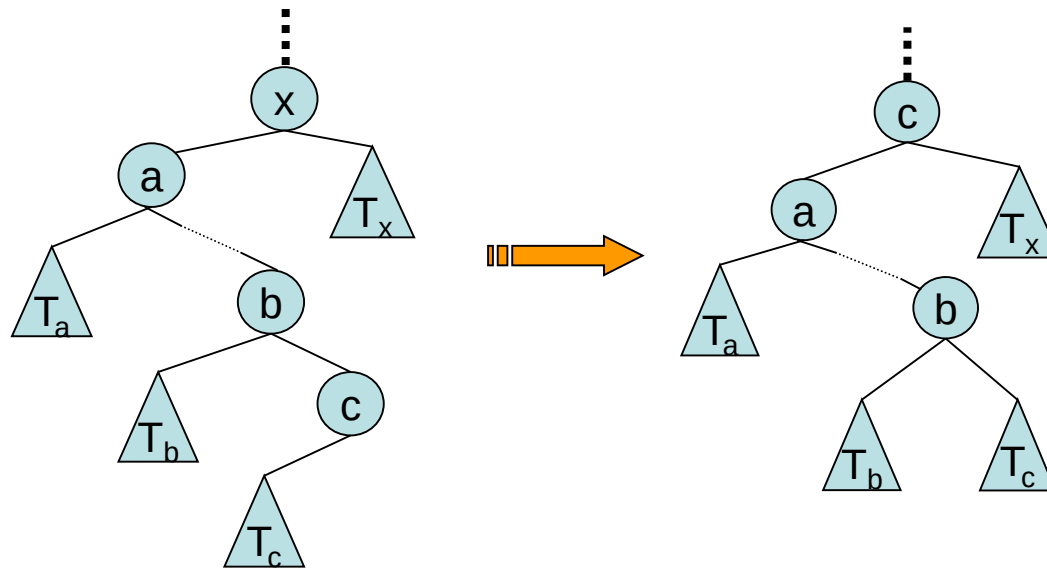
□ 삭제

- 앞 노드: 바로 삭제
- 자식이 하나인 노드: 해당 노드는 삭제하고 그 자리에 자식 노드를 갖다 놓는다
- 자식이 둘인 노드: 앞 노드나 자식이 하나인 노드의 삭제의 문제로 변환



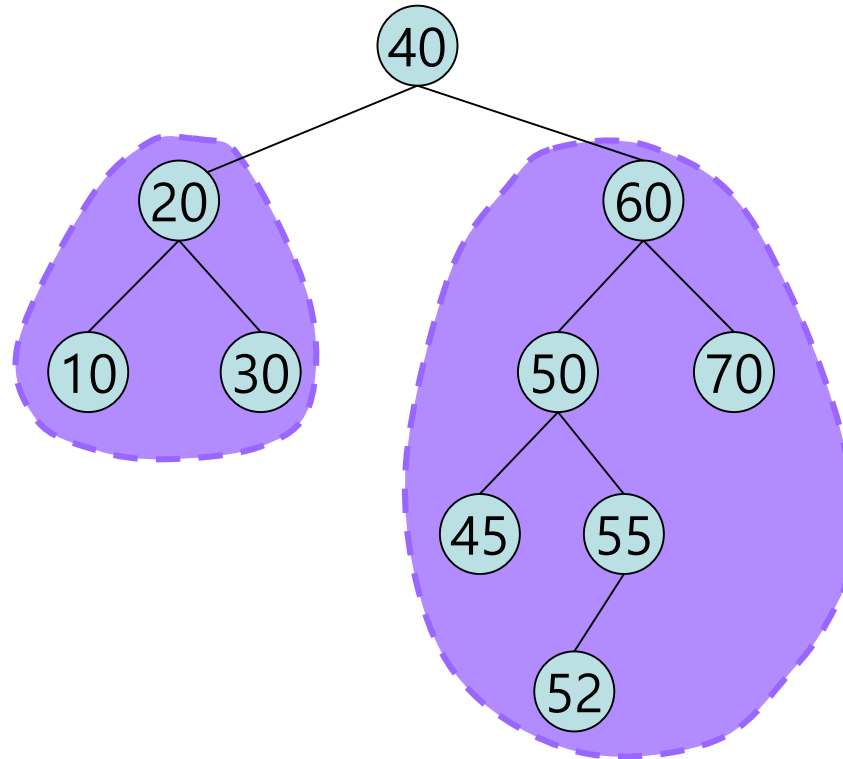
□ 내부 노드 X의 삭제

- X의 왼쪽 부트리에서 가장 큰 값을 갖는 노드로 대체
(또는 X의 오른쪽부트리에서 가장 작은 값을 갖는 노드로 대체)

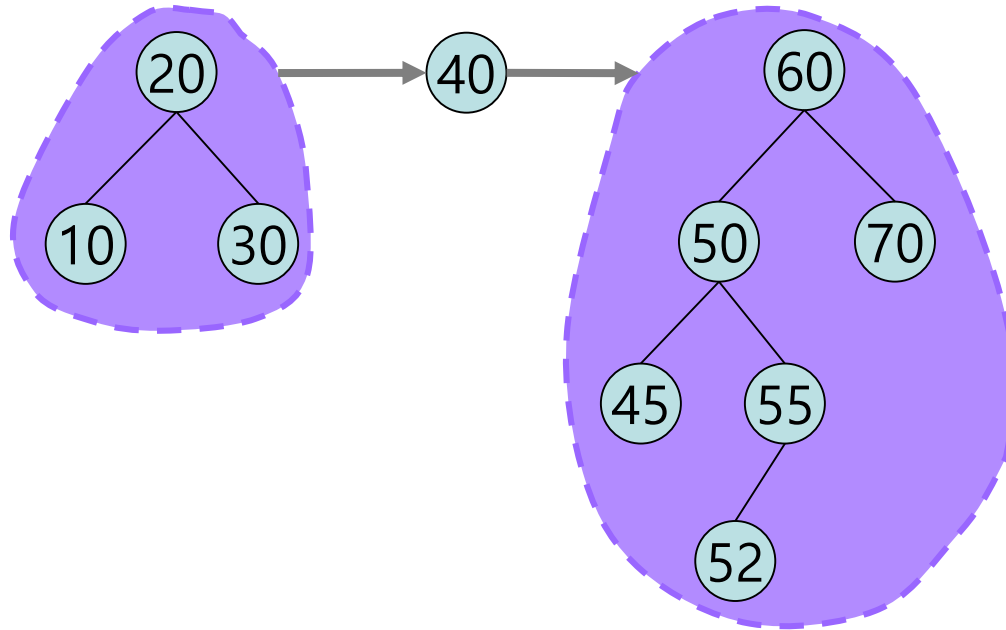


- 삭제의 시간복잡도: $O(\log n)$
 - 트리의 높이가 h 일 때, $O(h)$
 - 삽입이나 삭제가 무작위로 발생하면, $h = O(\log n)$

□ BST 에서 중위 탐색을 하면 ?



□ BST 에서 중위 탐색을 하면 ?



사전의 (Key, Object) 쌍을 위한

Class “DictionaryElement”

□ DictionaryElement: 공개함수

```
public class DictionaryElement<Key,Obj>
{
    public Element<Key,Obj> () ;
    public Element<Key,Obj> (Key aKey, Obj anObject) ;

    public Key      key () ;
    public void     setKey (Key newKey) ;

    public Obj      object () ;
    public void     setObject (Obj newObject) ;
}
```

□ DictionaryElement: 비공개 인스턴스 변수

```
public class DictionaryElement<Key, Obj>
{
    // 비공개 인스턴스 변수
    private Key    _key ;
    private Obj    _object ;
```

□ DictionaryElement: 생성자

```
public class DictionaryElement<Key,Obj>
{
    .....
    public DictionaryElement ()
    {
        this._key = null ;
        this._object = null ;
    }

    public DictionaryElement (Key givenKey, Obj givenObject) ;
    {
        this._key = givenKey ;
        this._object = givenObject ;
    }
}
```

□ DictionaryElement: Getter/Setter

```
public class DictionaryElement<Key,Obj>
{
    .....
    public Key  key () ;
    {
        return  this._key ;
    }

    public void  setKey (Key newKey) ;
    {
        this._key =  newKey ;
    }

    public Obj  object () ;
    {
        return  this._object ;
    }

    public void  setObject (Obj newObject) ;
    {
        this._object =  newObject ;
    }
}
```

사전 구현에 사용할
이진검색트리의 노드:

Class "BinaryNode"

□ BinaryNode의 공개함수

```
public class BinaryNode<T>
{
    .....

    public BinaryNode () ;
    public BinaryNode
        (T givenElement, BinaryNode<T> givenLeft, BinaryNode<T> givenRight) ;

    public T                element () ;
    public void              setElement (T newElement) ;

    public BinaryNode        left () ;
    public void               setLeft (BinaryNode<T> newLeft) ;

    public BinaryNode        right () ;
    public void               setRight (BinaryNode<T> newRight) ;
}
```

❑ BinaryNode: 비공개 인스턴스 변수와 생성자

```
public class BinaryNode<T>
{
    // 비공개 인스턴스 변수
    private T                _element ;
    private BinaryNode<T>    _left ;
    private BinaryNode<T>    _right ;

    public BinaryNode ()
    {
        this._element = null ;
        this._left = null ;
        this._right = null ;
    }

    public BinaryNode
        (T givenElement, BinaryNode<T> givenLeft, BinaryNode<T> givenRight)
    {
        this._element = givenElement ;
        this._left = givenLeft ;
        this._right = givenRight ;
    }
}
```


□ BinaryNode: Setter/Getter 의 구현

```
public T element()
{
    return this._element ;
}

public void setElement (T newElement)
{
    this._element= newElement ;
}

public BinaryNode left ()
{
    return this._left ;
}

public void setLeft (BinaryNode<T> newLeft)
{
    this._left = newLeft ;
}

public BinaryNode right ()
{
    return this._right ;
}

public void setRight (BinaryNode<T> newRight)
{
    this._right = newRight ;
}
```

Class "Dictionary"의 구현

□ Dictionary의 공개함수

■ Dictionary 객체 사용법

- public Dictionary();
- public boolean isEmpty ();
- public boolean isFull ();
- public int size();
- public boolean keyDoesExist (Key aKey) ;
- public Obj objectForKey (Key aKey) ;
- public boolean addKeyAndObject (Key aKey, Obj anObject) ;
- public Obj removeObjectForKey (Key aKey) ;
- public boolean replaceObjectForKey (Obj objectForReplace, Key aKey) ;
- public void clear () ;

□ Dictionary: 비공개 인스턴스 변수

```
public class Dictionary<Key extends Comparable<Key> , Obj>
{
    // 비공개 인스턴스 변수
    private int _size ;
    private BinaryNode<DictionaryElement<Key,Obj>> _root ;
```

- Class "Dictionary" 안에서 Key 객체 끼리의 비교가 필요.
- Generic class "Key" 를 대체 하는 실제 class 는 interface "Comparable" 을 구현하고 있어야 한다. 따라서 "compareTo()" 함수 역시 당연히 override 하여 구현하고 있어야 한다.

□ Dictionary의 생성자

```
public class Dictionary<Key extends Comparable<Key>,Obj>  
{
```

```
.....
```

```
// 생성자
```

```
public Dictionary()  
{  
    this._size = 0 ;  
    this._root = null ;  
}
```

□ Dictionary : 상태 알아보기

```
public class Dictionary<Key extends Comparable<Key>, Obj>
{
    .....
```

```
// 상태 알아보기
public boolean isEmpty()
{
    return (this._root == null) ;
}
```

```
public boolean isFull ()
{
    return false ;
}
```

```
public int size()
{
    return this._size ;
}
```

❑ Dictionary: keyExists() [recursive]

```

public boolean    keyExists (Key aKey)
{
    return this.keyExistsInTree (this._root, aKey) ;
}

private boolean  keyExistsInTree
    (BinaryNode<DictionaryElement<Key,Obj> > currentRoot, aKey)
{
    if ( currentRoot == null ) {
        return false ;
    }
    else {
        if ( aKey.compareTo(currentRoot.element().key()) == 0 ) {
            return true ;
        }
        else if ( aKey.compareTo(currentRoot.element().key()) < 0 ) {
            return this.keyExistsInTree (currentRoot.left(), aKey) ;
        }
        else {
            return this.keyExistsInTree (currentRoot.right(), aKey) ;
        }
    }
}

```

❑ Dictionary: `keyExists()` [nonrecursive]

```
public boolean    keyExists (Key aKey)
{
    boolean found = false ;
    BinaryNode currentRoot = this._root ;
    while ( (! found) && (currentRoot != null) ) {
        if (aKey.compareTo(currentRoot.element().key()) == 0 ) {
            found = true ;
        }
        else if ( aKey.compareTo(currentRoot.element().key()) < 0 ) {
            currentRoot = currentRoot.left() ;
        }
        else {
            currentRoot = currentRoot.right() ;
        }
    }
    return found ;
}
```


□ Dictionary: objectForKey()

```

public Obj objectForKey (Key aKey)
{
    boolean found = false ;
    BinaryNode<DictionaryElement<Key,Obj>> currentRoot = this._root ;
    while ( (! found) && (currentRoot != null) ) {
        if ( aKey.compareTo(currentRoot.element().key()) == 0 ) {
            found = true ;
        }
        else if ( aKey.compareTo(currentRoot.element().key()) < 0 ) {
            currentRoot = currentRoot.left() ;
        }
        else {
            currentRoot = currentRoot.right() ;
        }
    }
    if ( found ) {
        return currentRoot.element().object() ;
    }
    else {
        return null ;
    }
}

```

❑ Dictionary: addKeyAndObject() [recursive]

```
public boolean addKeyAndObject (Key aKey, Obj anObject)
{
    if (this._root == null) {
        this._root =
            new BinaryNode((new DictionaryElement(aKey, anObject)), null, null) ;
        this._size ++ ;
        return true ;
    } else {
        return addKeyAndObjectToSubtree (this._root, aKey, anObject) ;
    }
}
```

❏ Dictionary: addKeyAndObjectToSubtree()

```
private boolean addKeyAndObjectToSubtree
(BinaryNode<DictionaryElement<Key,Obj>> currentRoot, Key aKey, Obj anObject)
{
    if ( currentRoot.element().key().compareTo(aKey) == 0 ) {
        return false ;
    }
    else if ( aKey.compareTo(currentRoot.element().key()) < 0 ) {
        if ( currentRoot.left() == null ) { // We can add to the left
            DictionaryElement<Key,Obj> elementForAdd =
                new DictionaryElement<Key,Obj>(aKey, anObject);
            currentRoot.setLeft (
                new BinaryNode<DictionaryElement<Key,Obj>>(elementForAdd, null, null) ;
            this._size++ ;
            return true ;
        }
        else {
            return addKeyAndObjectToSubtree(currentRoot.left(), aKey, anObject) ;
        }
    }
    else {
        if ( currentRoot.right() == null ) { // We can add to the right
            DictionaryElement<Key,Obj> elementForAdd =
                new DictionaryElement<Key,Obj>(aKey, anObject);
            currentRoot.setRight (
                new BinaryNode<DictionaryElement<Key,Obj>>(elementForAdd, null, null) ;
            this._size++ ;
            return true ;
        }
        else {
            return addKeyAndObjectToSubtree(currentRoot.right(), aKey, anObject) ;
        }
    }
}
```

Dictionary: addKeyAndObject() [nonrecursive]

```

public boolean addKeyAndObject (Key aKey, Obj anObject)
{
    if ( this._root == null ) {
        DictionaryElement<Key,Obj> elementForAdd =
            new DictionaryElement<Key,Obj>(aKey, anObject);
        this._root= new BinaryNode<DictionaryElement<Key,Obj>> (elementForAdd, null, null) ;
        this._size ++ ;
        return true ;
    }
    BinaryNode<DictionaryElement<Key,Obj>> current = this._root ;
    while (aKey.compareTo(current.element().key()) != 0) {
        if ( aKey.compareTo(current.element().key()) < 0 ) {
            // We should go down to the left subtree.
            if ( current.right() == null ) { // No left subtree. We can add to the left of "current".
                DictionaryElement<Key,Obj> elementForAdd =
                    new DictionaryElement<Key,Obj>(aKey, anObject);
                BinaryNode<DictionaryElement<Key,Obj>> addedNode =
                    new BinaryNode<DictionaryElement<Key,Obj>> (elementForAdd, null, null) ;
                current.setRight(addedNode) ;
                this._size++ ;
                return true ;
            }
            current = current.left() ;
        }
        else {
            // We should go down to the right subtree.
            if ( current.right() == null ) { // No right subtree. We can add to the right of "current".
                DictionaryElement<Key,Obj> elementForAdd =
                    new DictionaryElement<Key,Obj>(aKey, anObject);
                BinaryNode<DictionaryElement<Key,Obj>> addedNode =
                    new BinaryNode<DictionaryElement<Key,Obj>> (elementForAdd, null, null) ;
                current.setLeft(addedNode) ;
                this._size++ ;
                return true ;
            }
            current = current.right() ;
        }
    }
    return false;
}

```

□ Dictionary: removeObjectForKey() (ver.1)

```

public Obj removeObjectForKey (Key aKey)
{
    Obj removedObject = null ;
    if ( this._root == null ) {
        return null ;
    } else ( aKey.compareTo(this._root.element().key()) == 0 ) {
        removedObject = this._root.element().object() ;
        if ( (this._root.left() == null) && ( this._root.right() == null) ) { // root만 있는 tree
            this._root = null ;
        }
        else if ( this._root.left() == null ) { // root의 left tree 가 없다
            this._root = this._root.right() ;
        } else { // root의 right tree 가 없다
            this._root = this._root.left() ;
        }
        else { // child의 left tree, right tree가 모두 있다
            this._root.setElement(removeRightMostElementOfLeftTree(this._root));
        }
        this._size -- ;
        return removedObject ;
    }
    else {
        return removeObjectForKeyFromSubtree (this._root, givenKey) ;
    }
}

```

□ Dictionary: **removeRightMostElementOfLeftTree()**

```
private DictionaryElement<Key,Obj> removeRightMostElementOfLeftTree (BinaryNode currentRoot)
{
    // 현재의 currentRoot 의 원소를 대체할 원소인,
    // 왼쪽 트리의 가장 오른쪽 노드의 원소를 삭제하여 얻는다.
    // call 되는 시점에, currentRoot.left() 는 null 이 아니다

    BinaryNode<DictionaryElement<Key,Obj>> leftOfCurrentRoot = currentRoot.left() ;
    if ( leftOfCurrentRoot.right() == null ) {
        currentRoot.setLeft (leftOfCurrentRoot.left()) ;
        return leftOfCurrentRoot.element() ;
    }
    else {
        BinaryNode<DictionaryElement<Key,Obj>> parentOfRightMost = leftOfCurrentRoot ;
        BinaryNode<DictionaryElement<Key,Obj>> rightMost = leftOfCurrentRoot.right() ;
        while ( rightMost.right() != null ) {
            parentOfRightMost = rightMost ;
            rightMost = rightMost.right() ;
        }
        parentOfRightMost.setRight (rightMost.left()) ;
        return rightMost.element() ;
    }
}
```

Dictionary: removeObjectForKeyFromSubtree() (recursive) [1]

```
private Obj removeObjectForKeyFromSubtree (BinaryNode currentRoot, Key aKey)
{
    // 이 시점에, currentRoot 는 null이 아니고, currentRoot의 key는 "aKey" 와 일치하지 않는다.
    if ( aKey.compareTo(currentRoot.element().key()) < 0 ) { // left subtree에서 삭제해야 한다
        child = currentRoot.left () ;
        if ( child == null ) {
            return null ;
        }
        else {
            if ( aKey.compareTo(child.element().key()) == 0 ) {
                Obj removedObject = child.element().object() ;
                if (child.left() == null && child.right() == null) { // child가 leaf
                    currentRoot.setLeft (null) ;
                }
                else if ( child.left() == null ) { // child 의 left tree가 없다
                    currentRoot.setLeft (child.right()) ;
                }
                else if ( child.right() == null ) { // child 의 right tree 가 없다
                    currentRoot.setLeft (child.left()) ;
                }
                else { // child의 left tree, right tree가 모두 있다
                    currentRoot.setElement (removeRightMostElementOfLeftTree(child)) ;
                }
                this._size -- ;
                return removedObject ;
            }
            else {
                return removeObjectForKeyFromSubtree (child, aKey) ;
            }
        }
    }
    else { // right subtree에서 삭제해야 한다
```

Dictionary: removeObjectForKeyFromSubtree() (recursive) [2]

```
private Obj removeObjectForKeyFromSubtree
(BinaryNode<DictionaryElement<Key,Obj>> currentRoot, Key aKey)
{
    if ( aKey.compareTo(currentRoot.element().key()) < 0 ) { // left subtree에서 삭제해야 한다
        .....
    }
    else { // right subtree에서 삭제해야 한다
        child = currentRoot.right() ;
        if ( child == null ) {
            return null ;
        }
        else {
            if ( aKey.compareTo(child.element().key()) == 0 ) {
                Obj removedObject = child.element().object() ;
                if ( child.left() == null && child.right() == null ) { // child가 leaf
                    currentRoot.setRight( null ) ;
                }
                else if ( child.left() == null ) { // child 의 left tree가 없다
                    currentRoot.setRight( child.right() ) ;
                }
                else if ( child.right() == null ) { // child 의 right tree 가 없다
                    currentRoot.setRight( child.left() ) ;
                }
                else { // child의 left tree, right tree가 모두 있다
                    currentRoot.setElement( removeObjectForKeyFromSubtree(child) ) ;
                }
                this._size -- ;
                return removedObject ;
            }
            else {
                return removeObjectForKeyFromSubtree( child, aKey ) ;
            }
        }
    }
}
```


Dictionary: `removeObjectForKeyFromSubtree()` (non-recursive) [1]

```
private Obj removeObjectForKeyFromSubtree (BinaryNode currentRoot, Key aKey)
{
    // 이 시점에, currentRoot 는 null이 아니고, currentRoot의 key는 aKey와 일치하지 않는다.
    do {
        if ( aKey.compareTo(currentRoot.element().key()) < 0 ) { // left tree에서 삭제 노드를 찾아야 한다
            child = currentRoot.left () ;
            if ( child == null ) {
                return null ;
            }
            if ( aKey.compareTo(child.element().key()) == 0 ) { // found
                Obj removedObject = child.element().object() ;
                if ( child.left() == null && child.right() == null ) { // child가 leaf
                    currentRoot.setLeft (null) ;
                }
                else if ( child.left() == null ) { // child 의 left tree가 없다
                    currentRoot.setLeft (child.right()) ;
                }
                else if ( child.right() == null ) { // child 의 right tree 가 없다
                    currentRoot.setLeft (child.left()) ;
                }
                else { // child의 left tree, right tree가 모두 있다
                    currentRoot.setElement (removeRightMostElementOfLeftTree(child)) ;
                }
                this._size -- ;
                return removedObject ;
            }
        }
        else { // right subtree에서 삭제 노드를 찾아야 한다
```

Dictionary: `removeObjectForKeyFromSubtree()` (non-recursive) [2]

```
private Obj removeObjectForKeyFromSubtree (BinaryNode<DictionaryElement<Key,Obj>> currentRoot, Key aKey)
{
    do {
        if ( aKey.compareTo(currentRoot.element().key()) < 0 ) { // left subtree에서 삭제해야 한다
            .....
        }
        else { // right subtree에서 삭제해야 한다
            child = currentRoot.right() ;
            if ( child == null ) {
                return null ;
            }
            if ( aKey.compareTo(child.element().key()) == 0 ) { // found
                Obj removedObject = child.element().object() ;
                if ( child.left() == null && child.right() == null ) { // child가 leaf
                    currentRoot.setRight (null) ;
                }
                else if ( child.left() == null ) { // child 의 left tree가 없다
                    currentRoot.setRight (child.right()) ;
                }
                else if ( child.right() == null ) { // child 의 right tree 가 없다
                    currentRoot.setRight (child.left()) ;
                }
                else { // child의 left tree, right tree가 모두 있다
                    currentRoot.right().setElement (removeRightMostNodeOfLeftTree(child)) ;
                }
                this._size -- ;
                return removedObject ;
            }
        }
        currentRoot = child;
    } while (true) ;
} // End of "removeObjectForKeyFromSubtree"
```

□ Dictionary: **replaceObjectForKey()**

```

public boolean replaceObjectForKey (Obj objectForReplace, Key aKey)
{
    boolean found = false ;
    BinaryNode<DictionaryElement<Key,Obj>> currentRoot = this._root ;
    while ( (! found) && (currentRoot != null) ) {
        if ( aKey.compareTo(currentRoot.element().key()) < 0 ) {
            currentRoot = currentRoot.left () ;
        }
        else if ( aKey.compareTo(currentRoot.element().key()) > 0 ) {
            currentRoot = currentRoot.right () ;
        }
        else {
            found = true ;
        }
    }
    if ( found ) {
        currentRoot.element().setObject (objectForReplace) ;
        return true ;
    }
    else {
        return false ;
    }
}

```

□ Dictionary: clear ()

```
public void clear ( )  
{  
    this._size = 0 ;  
    this._root = null ;  
}
```

실습: 사전의 성능 측정

□ 실습: 사전의 성능 측정

- 사전을 3 가지로 구현하여 성능을 측정한다.
- 구현 방법의 종류
 - Sorted Array (Increasing order)
 - Sorted Linked List (Increasing order)
 - Binary Search Tree
- 성능 측정 기능
 - 삽입
 - 검색
 - 삭제

□ 실습: 사전의 성능 측정

- 입력 :
 - 없음
 - 필요한 데이터는 프로그램에서 생성
- 출력 : 성능 측정 결과
 - 데이터 크기 변화에 따른 성능 측정 결과
 - 측정 단위: micro (10^{-6}) second
- 데이터 크기
 - 10000, 20000, 30000, 40000, 50000
- 데이터 생성
 - 모든 데이터는 중복되는 값이 없게 생성한다.
 - 오름차순: 0 부터 1씩 증가하는 데이터
 - 내림차순: (최대 크기 - 1) 부터 1 씩 감소하는 데이터
 - 무작위 데이터: 오름차순 데이터를 만든 다음, 각각의 값에 대해 Random number 를 생성하여 위치를 무작위로 바꾸어 사용한다.
 - ◆ 데이터의 값 자체를 random number 를 생성하여 사용할 수 없는 이유는?

□ 결과 화면[1] : 오름차순 데이터

■ SortedArray:

- 검색이 작은 이유?
- 삽입이 작은 이유?
- 삭제는 왜 큰지?
- 삭제가 다른 두 구현의 삽입에 비해 작은 이유?

■ SortedLinkedList:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 삽입이 SortedArray의 삭제보다 꽤 큰 이유? (7 배 정도?)

■ BinaryTree:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 결과가 SortedLinkedList와 유사하게 얻어지는 이유?

<<"Dictionary"의 성능 측정을 시작합니다.>>

<오름차순 데이터를 사용한 측정 (단위: mili second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	728	877	26972
20000	1572	2007	109530
30000	2685	2408	251138
40000	3122	3227	438648
50000	4493	4415	687421

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	171885	168137	387
20000	716519	857032	779
30000	1557409	1537019	1000
40000	2932034	3008850	1425
50000	4746520	4529352	2143

"DictionaryByBinaryTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	217141	187044	1691
20000	811290	745560	701
30000	1807138	1500126	1045
40000	2878727	2693346	1416
50000	4663363	4496717	1712

□ 결과 화면[2] : 내림차순 데이터

■ SortedArray:

- 삽입이 큰 이유?
- 삭제가 작은 이유?
- 삽입이 다른 두 구현의 삭제에 비해 작은 이유?

■ SortedLinkedList:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?

■ BinaryTree:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 삽입이 SortedLinkedList의 삭제와 유사하게 얻어지는 이유?
- 검색이 SortedLinkedList의 검색과 유사하게 얻어지는 이유?

<내림차순 데이터를 사용한 측정 (단위: mili second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	32514	760	569
20000	132609	1601	1192
30000	292742	2513	1840
40000	512736	3305	2496
50000	800985	4205	3168

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	410	187255	209335
20000	820	590586	658844
30000	1208	1906830	1842993
40000	1592	3123470	3076136
50000	2057	4879225	5000196

"DictionaryByBinaryTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	200398	209614	356
20000	813702	845654	739
30000	1910062	1757572	1080
40000	2957996	3086052	1440
50000	5261691	4634717	1738

□ 결과 화면[3] : 무작위 데이터

■ SortedArray:

- 삽입과 삭제가 비슷한 이유?
- 삭제가 다른 두 구현의 삽입에 비해 작은 이유?

■ SortedLinkedList:

- 검색이 삽입이나 삭제의 2 배 정도인 이유?
- 삽입과 삭제가 비슷한 이유?
- L SortedArray에 비해 꽤 큰 이유?

■ BinaryTree:

- 삽입/삭제/검색이 비슷한 이유?
- 다른 구현에 비해 많이 작은 이유?

<무작위 데이터를 사용한 측정 (단위: mili second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	16715	1421	14870
20000	64326	3356	58140
30000	150217	5459	139531
40000	253395	8120	230788
50000	400973	10055	353734

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	212496	510901	185847
20000	1099644	2403834	1037551
30000	2892668	5953355	2518965
40000	5493208	11071679	4787998
50000	8357133	16761459	7479491

"DictionaryByBinaryTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	1689	1781	1719
20000	4069	3810	4147
30000	6637	7056	7474
40000	8852	11602	9676
50000	12517	16289	12614

<<"Dictionary"의 성능 측정을 종료합니다.>>

“사전” [끝]

